

MODULE 5.1 · Concept 6 of 7 · NEW

Cognitive Architectures: Loops to State Graphs

Agent control flow is a designable artifact — LangGraph, AutoGen, and OpenAI Swarm

10 min

Duration

B3 – Apply

Bloom's

L3 – Advanced

Level

TH-000009

Prerequisite

AI-AG-IN-TH-000020

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — WHY

Beyond the ReAct Loop

Simple loops can't handle branching, merging, persistence, or multi-agent handoff

Why Simple Loops Aren't Enough

Real agents need branching, merging, persistence, and conditional flow

A ReAct loop is a fixed cycle: Thought → Action → Observation → Repeat.

But real agent tasks need:

- ↳ **Branching:** If user is angry → escalate to human; if calm → continue autonomously. Different paths for different conditions.
- ⊕ **Merging:** Three parallel searches complete → merge results into one analysis. Multiple inputs converge to one node.
- 💾 **Persistence:** Agent paused at step 7. Resume tomorrow from exactly where it left off. State saved and restored.
- ↪ **Handoff:** Research agent finishes → passes results to writing agent. Different agents for different phases.
- 🔄 **Cycles with exit:** Retry loop: try → fail → reflect → retry. But exit after 3 attempts. Loops with conditional termination.

The Evolution: Reactive → ReAct → State Machine → Graph

Four generations of agent control flow

	Architecture	Control Flow	State Management	Example
Gen 1	Reactive	IF→THEN rules	None	Thermostat, spam filter
Gen 2	ReAct Loop	Think→Act→Observe cycle	Context window	LangChain agent, ChatGPT
Gen 3	State Machine	Nodes + transitions	Typed state dict	Finite automata agents
Gen 4	State Graph	Nodes + conditional edges + cycles	Persistent + checkpointed	LangGraph, AutoGen

Key insight: Agent control flow is itself a designable artifact. You don't just pick 'ReAct' — you architect a graph that defines exactly how your agent thinks, acts, branches, and terminates.

Agent Control Flow as a Designable Artifact

You architect the graph — the graph controls the agent

In a ReAct agent, control flow is implicit — the LLM decides what to do next at every step.

In a graph-based agent, control flow is explicit — YOU define the nodes, edges, and conditions.

Implicit (ReAct)

LLM decides at every step:

'Should I search? Calculate? Answer?'

Pros: Flexible, adapts to anything

Cons: Unpredictable, hard to debug,
may take unexpected paths

Explicit (Graph)

Developer defines the graph:

'Node A → if angry: Node B, else: Node C'

Pros: Predictable, debuggable, testable

Cons: Less flexible, requires upfront design,
new paths need graph changes

SECTION 2 — WHAT

Three Modern Architectures

LangGraph, AutoGen, and OpenAI Swarm — different approaches to the same problem

LangGraph: Agent as a Stateful Graph

Nodes, edges, conditional edges, and cycles — the most flexible architecture

Core Concepts:

State

A TypedDict that carries all information between nodes. Every node reads and writes to shared state. Example: {messages: [], plan: [], results: {}}

Nodes

Python functions that process the state. Each node does one thing: call an LLM, execute a tool, route to next node. Example: `def search_node(state) → updated_state`

Edges

Fixed connections between nodes. After node A always runs, go to node B. Example: `search_node → analyze_node`

Conditional Edges

Dynamic routing based on state. If `sentiment='angry'` → `escalate_node`, else → `continue_node`. This is where branching happens.

LangGraph = state + nodes + edges + conditional edges + cycles. It's a directed graph where the agent's behavior is the graph structure.

LangGraph Core: State, Nodes, Edges, Conditional Edges

Definitions with code snippets

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List

# 1. Define State
class AgentState(TypedDict):
    messages: List[str]
    plan: List[str]
    current_step: int

# 2. Define Nodes (functions)
def planner(state): ... # Generate plan from goal
def executor(state): ... # Execute current step
def checker(state): ... # Check if goal is met

# 3. Build Graph
graph = StateGraph(AgentState)
graph.add_node("plan", planner)
graph.add_node("execute", executor)
graph.add_node("check", checker)
graph.add_edge("plan", "execute")
graph.add_conditional_edges("check", lambda s: "execute" if not s["done"] else END)
```

LangGraph Worked Example: Customer Support Router

A 3-node graph with conditional routing



This graph is deterministic: the conditional edge ('angry?') always produces the same routing for the same input. Compare to ReAct: the LLM might route differently each time based on subtle wording differences.

AutoGen + OpenAI Swarm: Alternative Architectures

Two more approaches to agent control flow

AutoGen (Microsoft)

Multi-agent conversation patterns.
Agents are chat participants that take turns.

- AssistantAgent: LLM-powered responder
- UserProxyAgent: executes code, human proxy
- GroupChat: multiple agents discuss a topic

Control flow = conversation protocol.
Agents negotiate by exchanging messages.
Best for: debate, brainstorming, review flows.

OpenAI Swarm

Lightweight agent handoff.
Agents pass control to each other.

- Agent: has instructions + tools
- Handoff: agent A → agent B
- Context: carried across handoffs

Control flow = handoff chain.
Minimal overhead — no orchestrator.
Best for: routing, triage, pipelines.

Comparing Architectures: LangGraph vs. AutoGen vs. Swarm

Choose based on control needs, not framework popularity

Dimension	LangGraph	AutoGen	OpenAI Swarm
Control model	State graph (explicit)	Conversation (emergent)	Handoff chain
State management	TypedDict + checkpoints	Conversation history	Context passing
Complexity	Medium–High	Medium	Low
Best for	Complex workflows, branching	Multi-agent debate/review	Simple routing/triage
Persistence	Built-in checkpointing	External (manual)	None (stateless)
Production readiness	High (LangChain ecosystem)	Medium	Experimental

SECTION 3 — WHEN

Choosing the Right Architecture

Match architecture complexity to task complexity

State Persistence and Checkpointing

Save agent state mid-execution — resume later without starting over

LangGraph supports built-in state persistence:

```
from langgraph.checkpoint import MemorySaver

checkpointer = MemorySaver() # Or SqliteSaver, PostgresSaver
graph = workflow.compile(checkpointer=checkpointer)

# Run until step 5, then pause
result = graph.invoke(input, config={"thread_id": "abc123"})

# Later: resume from exactly where we left off
result = graph.invoke(None, config={"thread_id": "abc123"})
```

Use cases: long-running research tasks (pause overnight), human-in-the-loop (pause for approval), crash recovery.

Decision Framework: When to Use What

Match architecture to task complexity

Simple single-step lookup	Just call the LLM	No loop needed. Overhead of any agent framework is wasted.
Multi-step reasoning (3–10 steps)	ReAct loop	Standard Thought→Action→Observation cycle handles this well.
Complex workflow with branching	LangGraph	Need conditional edges, state persistence, explicit control flow.
Multi-agent debate/review	AutoGen	Multiple agents exchanging messages, taking turns, reaching consensus.
Simple agent-to-agent routing	OpenAI Swarm	Lightweight handoff. Triage agent → specialist agent.

Start simple (ReAct). Add graph structure only when you need branching, persistence, or multi-agent coordination.

Key Takeaways

The essential points from this concept

- Agent control flow is a designable artifact** — not just a fixed loop, but a graph you architect.
- Evolution: reactive → ReAct → state machine → state graph** — each adds control capability.
- LangGraph: explicit state graph** with nodes, conditional edges, cycles, and persistence.
- AutoGen: conversation patterns.** Swarm: lightweight handoff. Each fits different use cases.
- Start with ReAct, upgrade to LangGraph** when you need branching, persistence, or checkpointing.

Next: AI-AG-IN-CD-000002 — Planning System Implementation (Coding Lab)

AI-AG-IN-TH-000020 Complete

Cognitive Architectures: Loops to State Graphs

Concept 6 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi